

Static ASL Sign Recognition - IAA Project

Simão Santos nº119042, Gabriel Gonçalves nº119550
DETI- Universidade de Aveiro

I. INTRODUCTION

Sign language recognition felt like a good project fit as it's a real problem with some genuine difficulty, and it sits nicely in the space of computer vision for accessibility. We started with the intent to classify based on video, but later narrowed the scope to static ASL letters images only, not because continuous signing isn't interesting, but because it kept things realistic for the time we had. Even with this simpler setup there are still real challenges to deal with: many letters look very similar, people sign at different angles and distances, and lighting varies a lot between our test environments.

The task is eleven-class classification: given an image of a hand sign, predict one of A, B, C, D, E, F, G, H, I, L or Y. We picked these eleven partly for variety (some pairs are visually very close, others are not). We excluded dynamic letters such as J and Z because the model is frame-based.

II. RELATED WORK

Most recent work on static sign recognition follows one of two directions: learning directly from images with convolutional neural networks, or extracting hand structure first and classifying a lower-dimensional representation. CNN-based approaches are natural for ASL alphabet recognition because they can learn visual features from the image itself; for example, deep CNN and Inception-style models have been used successfully for gesture and image classification tasks [1], [2]. The downside is that they can also learn background, lighting and dataset-specific cues when the training data is not diverse enough, which was exactly the failure mode we observed in our transfer-learning prototype.

The alternative is to rely on hand landmarks and treat recognition as a geometric classification problem. MediaPipe Hands makes this practical by providing real-time hand tracking and 21 landmark positions from RGB input [3]. Once the image is reduced to normalized geometric features, classical classifiers such as SVMs become a reasonable option: they work well on structured feature vectors, are fast to train, and expose failure patterns more clearly than a large CNN [4]. Our final pipeline follows this second direction, using the public ASL Alphabet dataset only as an initial starting point before moving to self-collected data because the controlled Kaggle images did not represent our live test conditions [5].

III. DATA AND PREPROCESSING

A. Origins and the Bias Problem

We started with a mix of public ASL image datasets and images we collected ourselves. From the beginning we knew

public datasets could be a risk: they tend to have clean, consistent backgrounds, similar lighting and not much variation in who's signing. That concern turned out to be justified. Classes A, B and C came from one dataset, while Y, L and F came from another. On a random split from the same data everything looked fine, but as soon as we tested with photos we took ourselves, the results fell apart for anything that wasn't A, B or C. The model had basically learned to recognize the dataset rather than the signs.

B. Final Feature Pipeline

For the final version we dropped the public datasets and built our own from scratch, taking pictures ourselves in different environments and with the hand in different orientations. Instead of using raw images, we switched to geometric features extracted by MediaPipe. It detects the hand and gives us 21 3D landmarks; we shift everything so the wrist is at the origin and scale by the wrist-to-middle-MCP distance so the features are size-invariant regardless of how close the hand is to the camera. We throw out frames where the hand is unrealistically small (<5% of the image) or large (>80%). From those normalised landmarks we compute 23 features — fingertip distances, finger curl, palm tilt and rotation, thumb angle, spread angles, and vertical variance across fingertips — and store them in `landmarks.csv`.

The dataset ended up with 20,133 training samples and 4,001 validation samples, kept in completely separate files. We were very deliberate about not letting the validation set be a random slice of the same source — we had already seen how badly that can mislead you. Table I shows the per-class breakdown.

Table I
LANDMARK DATASET CLASS DISTRIBUTION.

Label	A	B	C	D	E	F	G	H	I	L	Y
Train	1654	2108	1273	1699	2032	1911	2014	1998	1665	1998	1781
Val.	250	377	221	297	311	411	268	417	628	376	445

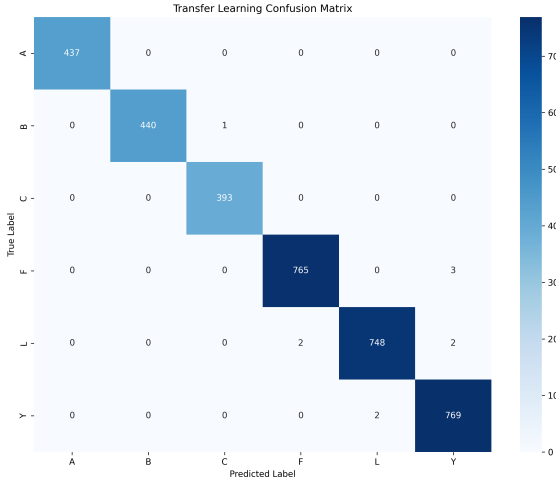
Beyond the raw sample counts, the main data-quality issue was not class frequency but diversity. The final training set is reasonably balanced for a self-collected dataset. The extracted features are all hand-relative geometric quantities, so their values are comparable across images after wrist translation and scale normalisation; this reduces camera-distance effects but does not remove errors caused by landmark drift or self-occlusion.

IV. DEVELOPMENT HISTORY

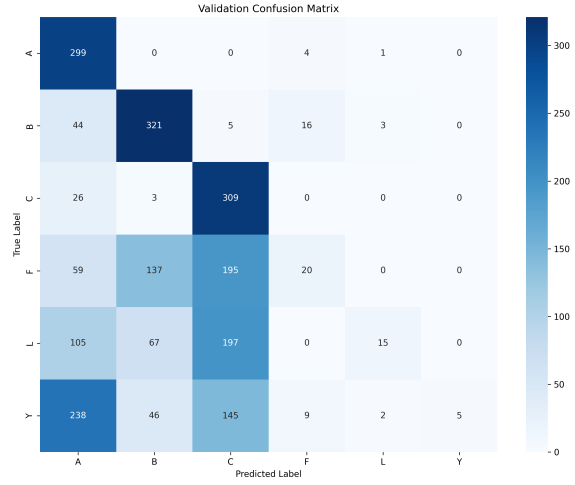
A. Transfer Learning with InceptionV3

Our first attempt at this used transfer learning on cropped hand images. The idea was straightforward: use MediaPipe to find the hand, crop to that region, resize to 299×299 and feed into InceptionV3 with pretrained weights. We trained the model by fine-tuning the last 30 layers with a smaller learning rate. We used augmentation (brightness, contrast, saturation, zoom, small shifts) on training data only, and skipped horizontal flipping since mirroring a hand can change which sign it is.

The in-distribution results looked good (Fig. 1(a)), but when we tested with photos we took ourselves the story was very different (Fig. 1(b)) — only A, B and C still worked. One thing we also noticed was that when a face was visible behind the hand the model got confused more often, which made us think it was paying too much attention to the background. On top of the bias issues and a clearly overfit model, training was just slow and painful to iterate — fine-tuning runs took long enough that trying things freely wasn't really practical. Between all of that, we decided to move on to something simpler.



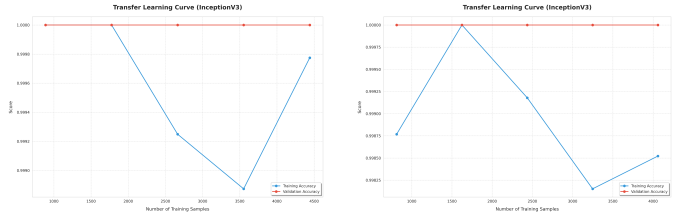
(a) Validation on a split from the same processed dataset.



(b) Validation using images we captured ourselves.

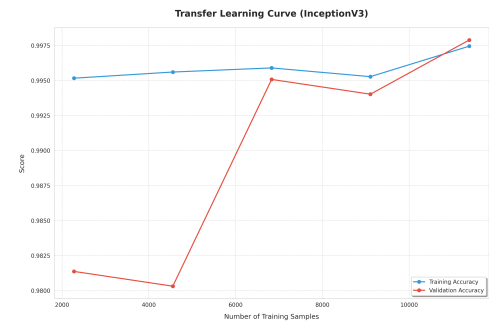
Figure 1. Transfer-learning confusion matrices. The good controlled-set result (a) did not survive the move to our own live images (b), which exposed the dataset-bias problem directly.

We also looked at learning curves, because the first ones (Fig. 2(a)) were weirdly flat and we weren't sure why. Changing the split from 70/20/10 to 80/20 didn't help much (Fig. 2(b)). Eventually we figured A, B and C are just too different from each other for a learning curve to tell you anything interesting. Once we added Y, F and L — which have more visual overlap with the existing classes — the curve finally started behaving in a way that made sense (Fig. 2(c)): small datasets overfit, larger ones generalise better. We also caught a normalisation bug here: the rescaling layer needed to go before the backbone, not after, to match what InceptionV3 actually expects.



(a) A/B/C only.

(b) New split.



(c) After adding visually closer classes.

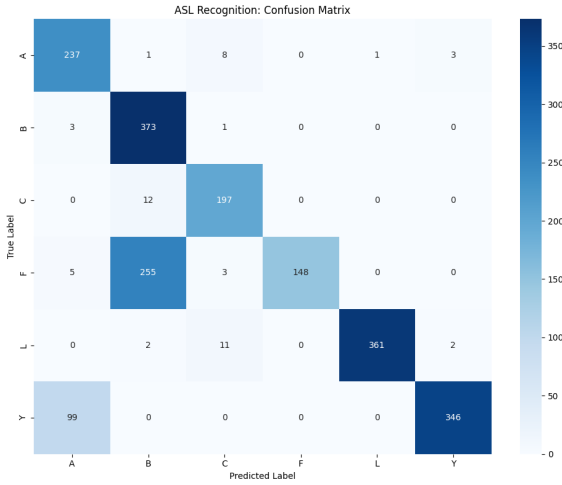
Figure 2. Transfer-learning learning curves. The curve only became informative once we added classes with visual overlap, showing why class selection matters for diagnosing model behaviour.

Even after these corrections the prototype remained hard to interpret and still leaned on dataset-specific patterns. That led us to try a completely different approach.

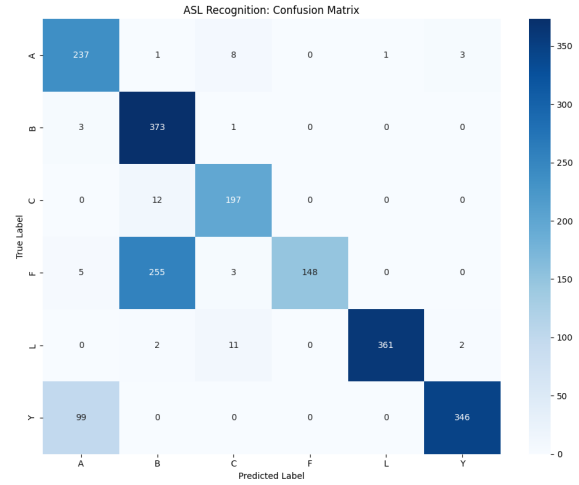
B. SVM Iterations and the Feature Bug

With the SVM we switched entirely to MediaPipe landmarks and hand-crafted geometry instead of raw pixels. This worked much better for our purposes: the features were compact, training took seconds, and when something was wrong we could actually look at the features and figure out why. First results were suspiciously good — around 99% accuracy and F1 — but live tests quickly showed B and F being mixed up, and that wasn't showing up in the confusion matrix at all. That told us the validation set wasn't capturing the real difficulty of the problem.

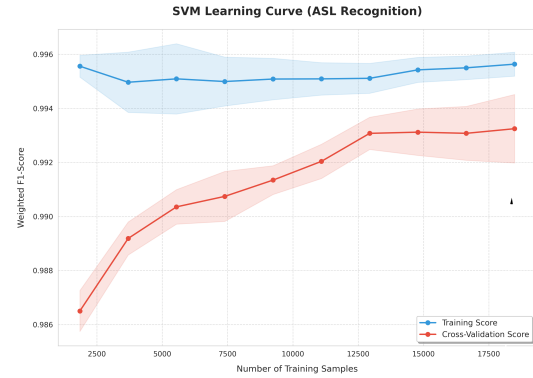
There were two issues. First, adding new classes had thrown off the class balance, so we rebalanced and added `class_weight=balanced`. Second, and more importantly, we found a bug: the feature extraction was silently dropping some distances between the index finger and thumb, which is exactly what tells B and F apart. Once we fixed that and built a validation set from our own live photos, the confusion matrix finally matched what we were seeing in practice. That version (v3) landed around 95% accuracy and 0.93 macro F1 — lower numbers, but ones we actually trusted. Figure 3 shows the confusion matrix and learning curve.



(a) SVM v3 confusion matrix.



(a) SVM v3 confusion matrix.



(b) SVM v3 learning curve.

Figure 3. SVM results after restoring the missing thumb-index features and switching to a more realistic validation set. These numbers felt much more credible than the earlier 99% figures.

V. FINAL MODEL AND EVALUATION STRATEGY

The final model is an RBF-kernel SVM. SVMs are sensitive to feature scale, so the normalisation step from the preprocessing pipeline matters a lot here — without it, features with larger numerical ranges would dominate. We handle class imbalance with `class_weight=balanced` and ran `GridSearchCV` over $C \in \{0.1, 1, 10, 100\}$ and $\gamma \in \{\text{scale}, \text{auto}, 0.001, 0.01, 0.1\}$ with 5-fold cross-validation scored on weighted F1.

For metrics we report accuracy, balanced accuracy, macro F1 and a normalised confusion matrix. We used macro F1 specifically because raw accuracy hid real problems for us earlier — it was easy to get a high number while the model was failing on specific classes. We also computed a new learning curve to track macro F1 across increasing training set sizes, which gave us a more grounded way to talk about overfitting and underfitting than a single validation score.

VI. RESULTS AND DISCUSSION

Fig. 4 shows the final confusion matrix as percentages. Most signs do well. C, E, F, B and H all sit above 98% recall. The

weaker ones are more interesting: A gets to 88% and mostly gets confused with I; D hits 84.51% with errors going toward F and A; L reaches 93.88% with some confusion toward D. These make sense if you think about the geometry — A and I are both closed fists, D and F share a lot of index-finger shape, and L and D can look similar depending on the angle. The fact that these confusions show up at all is actually a good sign — earlier confusion matrices looked perfect because the validation data was basically the same as the training data.

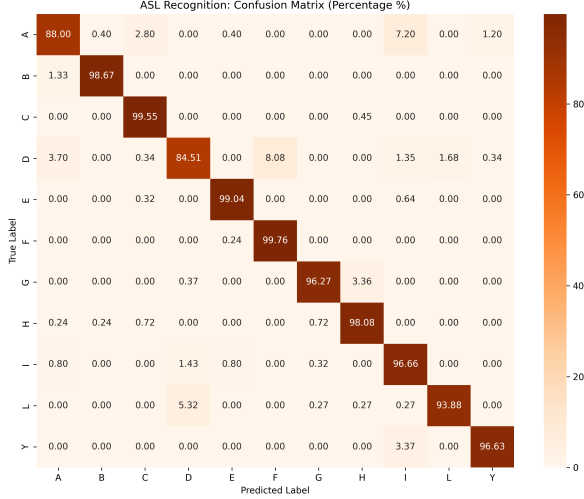


Figure 4. Final SVM percentage confusion matrix on the validation set. The remaining errors fall in geometrically sensible places.

The learning curve in Fig. 5 is where the overfitting question becomes clearest. Training macro F1 sits near 0.99 throughout, while cross-validation F1 stays around 0.91–0.93. That consistent gap is an early sign of overfitting: the model is fitting the training data very well but not generalising quite as cleanly to data it hasn’t seen. It’s not severe — the validation score does tick upward as more data is added, and performance doesn’t collapse at small sample sizes, so we’re not dealing with underfitting either. The model has enough capacity; what it’s missing is more varied data. The current training set doesn’t cover enough different angles, lighting conditions and signing styles, so the model memorises what it’s seen rather than learning something fully general. Adding more diverse samples would likely close that gap more than any amount of hyperparameter tuning.

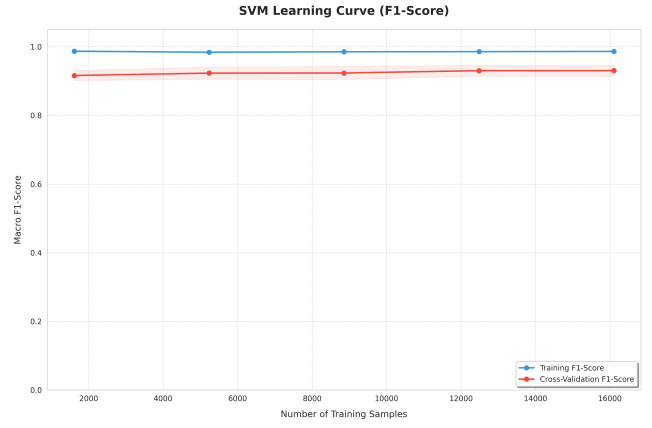


Figure 5. SVM learning curve generated by `learning_curve_svm.py`. The gap between training and validation scores suggests mild overfitting rather than underfitting.

VII. ROTATION ROBUSTNESS

Beyond the standard metrics, we wanted to see how the model behaves when the hand is rotated. We measured failure rate while rotating each sign horizontally, grouping samples into 10-degree buckets. This was a way to find failure modes that the confusion matrix would never show.

As Fig. 6 shows, rotation doesn’t hurt everything equally. A and B stay robust throughout, probably because their key features don’t change much under moderate roll. C, D, F, I and Y start breaking down at higher angles. E and L go wrong pretty quickly after any significant rotation, while G and H become unreliable the moment the specific training angle changes.

It is important to clarify how this rotation is being calculated and the implications it has on the metrics taken. To compute the rotation of the hand we use the MediaPipe’s pinky, thumb and base of the wrist nodes to form a plane, and the rotation that is tracked is the one from the defined plane. This means that in higher angles, where MediaPipe itself struggles to map the points onto the hand, the rotation tracking also deteriorates. In the specific case of the G and H signs the missing data is due to the fact that, in our script we defined the starting angle as with the palm of the hand towards the camera, as the G and H are the only labels in our project that have the back of the hand face the camera, the script incorrectly mapped data that was supposed to be related to 0° to 110°.

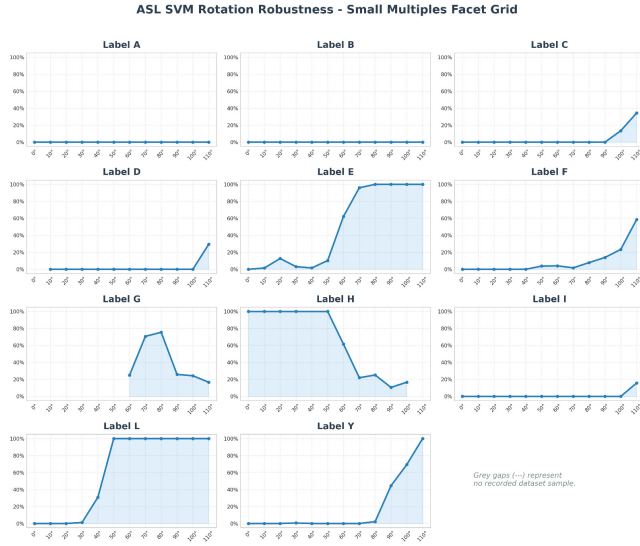


Figure 6. Failure rate per class as horizontal rotation increases. A and B are remarkably robust; E and L degrade quickly.

The most likely explanation is that profile views and self-occlusion mess with the landmark positions MediaPipe outputs. When a finger hides behind the palm, the estimated keypoint drifts away from where it actually is, and the SVM just runs with that bad input. It’s one of the trade-offs of the landmark approach: it’s faster and more interpretable, but it’s only as good as the tracker feeding it. Fig. 7 breaks this down per class and angle bucket.

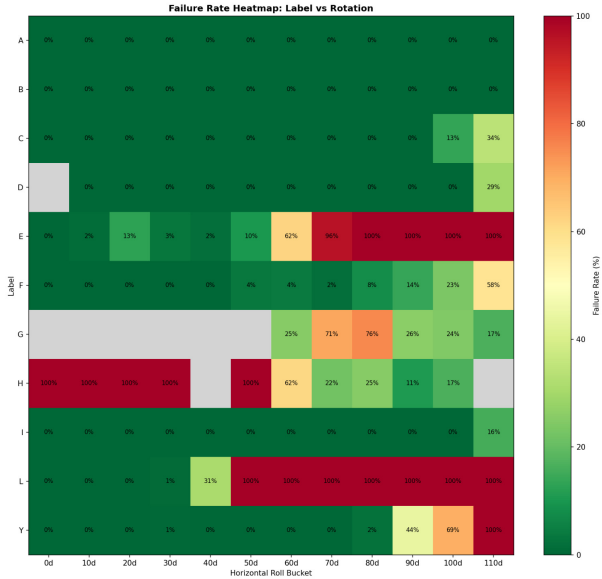


Figure 7. Failure-rate heatmap by class and rotation bucket. Grey cells indicate angle ranges with no collected samples for that class.

VIII. RESPONSIBLE AND ETHICAL CONSIDERATIONS

With only eleven classes and no notion of grammar or movement, this system is far from a real ASL translator, and it

shouldn’t be treated as one. It recognizes isolated static letters under controlled conditions, and that’s it.

Even for this limited use case, there are things worth being careful about. Hand images are personal data, and any real deployment would need to be upfront about what’s being captured and stored. Since we built the final dataset ourselves, the data is not diverse — it mostly reflects our own hands, our cameras and our lighting. Someone with a different hand size, skin tone, signing style or disability could get much worse results. The numbers we report should be read as showing what the model can do in our specific setup, not as a general performance guarantee.

IX. CRITICAL REFLECTION AND CONCLUSIONS

This project demonstrated that complex deep learning architectures are not always the optimal choice for every problem. While our initial InceptionV3 pipeline offered high capacity, its vulnerability to dataset bias and slow training cycles made iterating impractical. Transitioning to a simpler, hand-crafted feature pipeline paired with an RBF-kernel SVM drastically improved interpretability, reduced training times to seconds, and decoupled the classification task from environmental background noise.

Our final framework achieved a robust macro F1-score of 0.91–0.93 on realistic live test images, though its reliability remains strictly bound to the tracking quality of the upstream MediaPipe layout under extreme rotations. Ultimately, the project underscores that prioritizing data transparency, structural feature design, and rigorous out-of-distribution validation yields a far more trustworthy system than simply relying on model complexity.

ACKNOWLEDGEMENT OF AI TOOLS

During the development of this project, artificial intelligence tools—specifically Large Language Models including Google Gemini Flash and Codex 5.4—were utilized to assist with code correction, rapid prototyping, and hyperparameter tuning. The core architectural choices, experimental designs, and final project decisions remained entirely the responsibility of the authors, guided by the advice of the professor. AI was also used to help polish this report’s language and spot missing data from it.

REFERENCES

- [1] L. Pigou, A. van den Oord, S. Dieleman, M. Van Herreweghe, and J. Dambre, “Using deep convolutional networks for gesture recognition in American Sign Language,” arXiv:1710.06836, 2017.
- [2] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, “Rethinking the Inception architecture for computer vision,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, 2016, pp. 2818–2826.
- [3] F. Zhang *et al.*, “MediaPipe Hands: On-device real-time hand tracking,” arXiv:2006.10214, 2020.
- [4] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [5] A. Nagaraj, “ASL Alphabet,” Kaggle, 2018. [Online]. Available: <https://www.kaggle.com/datasets/grassknoted/asl-alphabet>